

Sequential RISC-V Processor Design

Sanjana Punna Saikiran S Himangi Sahoo
2024102065 2024112007 2024112020

February 2026

Contents

Introduction	2
Stage 1: Instruction Fetch	3
Program Counter(PC):-	3
Instruction memory(IR):-	4
Stage 2: Instruction Decode	4
Control Unit:-	5
Registers:-	6
Immediate Generator:-	6
Stage 3: Execution	7
ALU Control Unit:-	7
The Arithmetic Logic Unit (ALU):-	8
Stage 4: Memory Access	10
Memory Access Behavior	10
Role of MemRead and MemWrite	11
Stage 5: Write Back	12
Implementation of Fibonacci Sequence Code	12

Introduction

A sequential processor (also known as a non-pipelined processor) executes every instruction step-by-step in a single processing unit. In this design, the entire instruction completes before the next one begins. Hence, at any moment in time, only one instruction is active inside the CPU.

Our processor only supports the following instructions: **add**, **sub**, **addi**, **and**, **or**, **ld**, **sd** and **beq**.

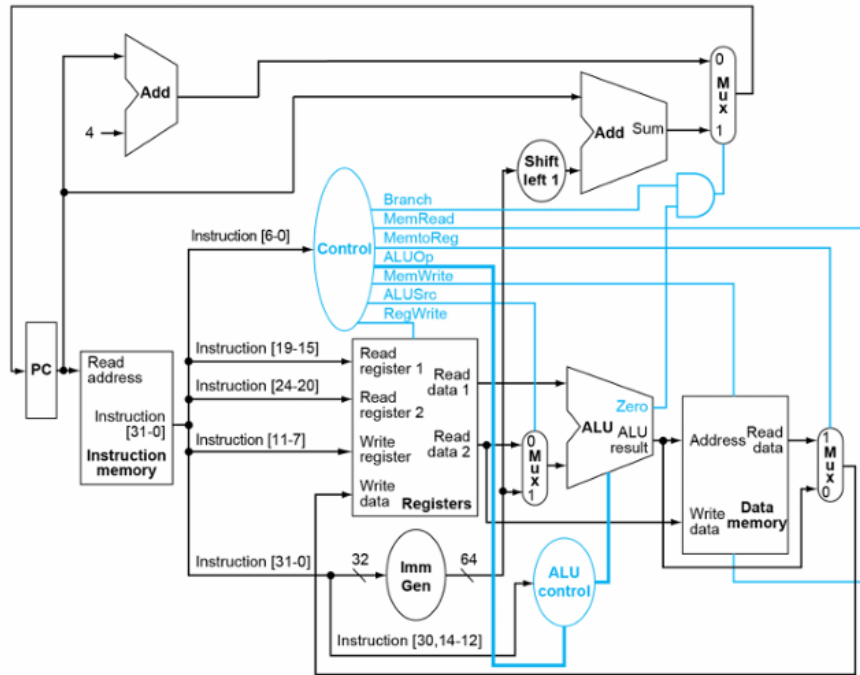


Figure 1: Sequential Datapath

Although the processor is sequential, each instruction still goes through the following five logical stages. The five stages are:

1. Instruction Fetch(IF)
2. Instruction Decode(ID)
3. Execute(EX)
4. Memory Access (MEM)
5. Write Back (WB)

Stage 1: Instruction Fetch

This stage is responsible for retrieving the next instruction from the memory.

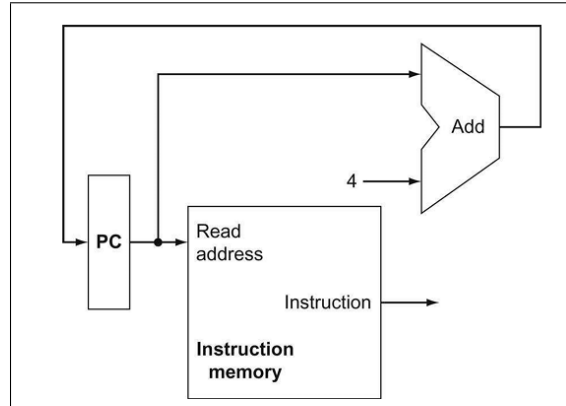


Figure 2: PC and Instruction memory

Program Counter(PC):-

- The Program Counter (PC) is a 64-bit register which holds the address of the instruction to be executed.
- It is basically a pointer which points to the address of the next executable instruction.
- The PC is updated by 4 after each 32-bit (4 bytes) instruction to point to the next instruction. This is done by using an adder.
- The PC is incremented to point to the next instruction.
- A multiplexer (MUX) selects between the next sequential instruction address ($PC + 4$) and the branch target address, depending on the branch control signal.

Verification:

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

PASS at time 10000 | PC = 0000000000000000
PASS at time 20000 | PC = 0000000000000004
PASS at time 30000 | PC = 0000000000000008
PASS at time 40000 | PC = 000000000000000c
PASS at time 50000 | PC = 0000000000000064

ALL TESTS PASSED
pc_tb.v:74: $finish called at 50000 (1ps)

```

Figure 3: Sample Testbench output of Program Counter (PC)

Instruction memory(IR):-

- The address of the next instruction is passed to the Instruction memory.
- It fetches the instructions from the memory and sends the 32-bit instruction to the next stage .i.e, the Instruction Decode Stage.

Verification:

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

PASS at addr 0 | Instruction = 00500113
PASS at addr 4 | Instruction = 00a00193
PASS at addr 8 | Instruction = 003100b3
PASS at addr 12 | Instruction = 40310133
PASS at addr 16 | Instruction = 0031f233
PASS at addr 20 | Instruction = 0041f2b3
PASS at addr 24 | Instruction = 00416333
PASS at addr 28 | Instruction = 003163b3
PASS at addr 32 | Instruction = 0012b023
PASS at addr 36 | Instruction = 0002b503
PASS at addr 40 | Instruction = 0062bc23
PASS at addr 44 | Instruction = 0182b583
PASS at addr 48 | Instruction = 00520463
PASS at addr 52 | Instruction = 00000063
PASS at addr 56 | Instruction = 00a086b3

ALL INSTRUCTION MEMORY TESTS PASSED
instruction_mem_tb.v:56: $finish called at 15000 (1ps)

```

Figure 4: Sample Testbench output of Instruction Memory

Stage 2: Instruction Decode

This stage decodes the fetched instruction to make the processor identify the required operands for execution, by extracting the opcode, register addresses, immediate values etc.

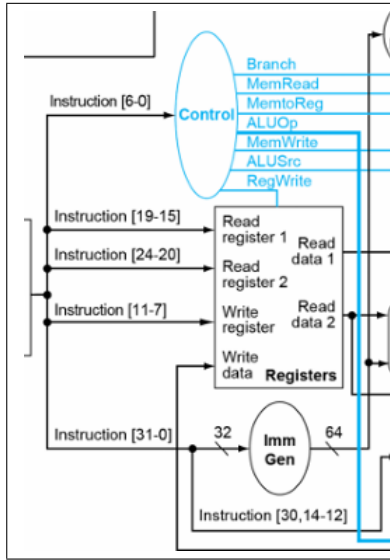


Figure 5: Instruction Decode

Control Unit:-

- The control unit decodes the instruction to determine its meaning - it receives the Opcode [6:0] and generates the corresponding control signals.
- The Opcodes are as follows :-

Instruction	Opcode
R-type	0110011
Load	0000011
Store	0100011
Branch	1100011

- Identifies the required registers and addressing modes.
- Selects the ALU operation or control signals needed for execution.

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

-----Control Unit Tests-----
PASS opcode=0110011
PASS opcode=0000011
PASS opcode=0100011
PASS opcode=1100011
control_unit_tb.v:66: $finish called at 4000 (1ps)

```

Figure 6: Sample Testbench output of Control Unit

Registers:-

- Bits [19:15] and [24:20] are used to read values from the Register File.
- Bits [11:7] are used to store result in the Register File after the operation is executed.

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

PASS at time 11000 | Read1: 0000000000000000 Read2: 0000000000000000
PASS at time 22000 | Read1: aaaabbbbccccddd Read2: 0000000000000000
PASS at time 33000 | Read1: 123456789abcdef0 Read2: aaaabbbbccccddd
PASS at time 44000 | Read1: 0000000000000000 Read2: aaaabbbbccccddd

ALL TESTS PASSED
reg_file_tb.v:133: $finish called at 44000 (1ps)

```

Figure 7: Sample Testbench output of Register

Immediate Generator:-

- It extracts the immediate values for load, store, I-type and B-type instructions.
- it converts the immediate values from 32-bit to 64-bit - it just extends the last bit till 64-bits.

	31	12	11					5	4			0
R	n/a											
I	SE<inst[31]						inst[31:20]					
S	SE<inst[31]						inst[31:25]				inst[11:7]	
B	SE<inst[31]						inst[7 30:25]				[11:8]	
U	inst[31:12]						0	0	0	0	0	0
J	SE<inst[31] inst[19:12] inst[20 30:25] [24:21]											

Figure 8: 32-bit Immediate value bits for different types of instruction

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```
---- I-TYPE TEST (addi x2, x0, -5) ----  
Immediate = ffffffffbb  
---- I-TYPE TEST (ld x1, 16(x0)) ----  
Immediate = 00000000000010  
---- S-TYPE TEST (sd x6, 24(x5)) ----  
Immediate = 00000000000018  
---- B-TYPE TEST (beq x1, x2, 8) ----  
Immediate = 00000000000008  
ImmGen_tb.v:41: $finish called at 40000 (1ps)
```

Figure 9: Sample Testbench output of Immediate Generator

Stage 3: Execution

This stage fetches the data from the registers and executes the operation decoded by the Instruction Decode Stage.

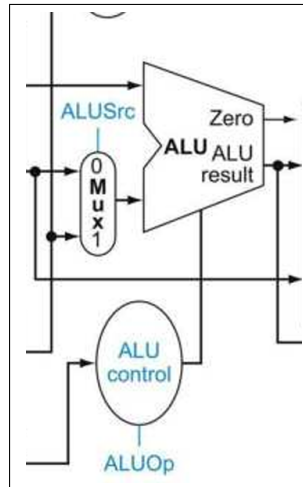


Figure 10: ALU Control and ALU

ALU Control Unit:-

- It fetches the ALUOp code from Control Unit to determine the type of instruction to be operated i.e. S-type, I-type, B-type etc.
- Based on instruction code [31:0], it determines which operation to perform i.e. Add, Subtract, AND, OR, by decoding the funct7 and funct3 field.

- Generates a 4-bit instruction which corresponds to which operation to execute.
- The 4-bit instructions with its corresponding function is as follows :-

ALU control lines	Function
0000	AND
0001	OR
0010	Add
0110	Subtract

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Figure 11: ALUOp, ALU control lines and its corresponding functions

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

----- ALU CONTROL TEST -----
ALUOp=00 (load/store) -> ALUControl = 0010 (Expected 0010)
ALUOp=01 (beq) -> ALUControl = 0110 (Expected 0110)
ADD -> ALUControl = 0010 (Expected 0010)
SUB -> ALUControl = 0110 (Expected 0110)
AND -> ALUControl = 0000 (Expected 0000)
OR -> ALUControl = 0001 (Expected 0001)
----- TEST COMPLETE -----
AluCtrl_tb.v:52: $finish called at 60000 (1ps)

```

Figure 12: Sample Testbench output of ALU Control Unit

The Arithmetic Logic Unit (ALU):-

- Receives the 4-bit instruction from ALU control Unit.
- Executes operations on the data fetched from registers.
- Operations may include arithmetic (add, subtract), logical (AND, OR), or address calculation (base + offset).

ALUOp		Funct7 field						Funct3 field				Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

Figure 13: ALU truth table

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

Test 1: A: 0000000000000000 B: 0000000000000000 Opcode: 0010 Result: 0000000000000000 Flags: C=0, D=0, Z=1
Test 2: A: 7fffffff00000000 B: 0000000000000001 Opcode: 0010 Result: 0000000000000000 Flags: C=0, D=1, Z=0
Test 3: A: 0000000000000000 B: 0000000000000000 Opcode: 0010 Result: 0000000000000000 Flags: C=1, D=1, Z=1
Test 4: A: ffffffff00000000 B: 0000000000000001 Opcode: 0010 Result: 0000000000000000 Flags: C=1, D=0, Z=1
Test 5: A: ffffffff00000000 B: 0000000000000000 Opcode: 0010 Result: ffffffff00000000 Flags: C=1, D=0, Z=0
Test 6: A: 7fffffff00000000 B: 0000000000000001 Opcode: 0010 Result: ffffffff00000000 Flags: C=0, D=1, Z=0
Test 7: A: 8000000000000000 B: ffffffff00000000 Opcode: 0010 Result: 7fffffff00000000 Flags: C=1, D=1, Z=0
Test 8: A: 0000000000000001 B: ffffffff00000000 Opcode: 0010 Result: 0000000000000001 Flags: C=1, D=0, Z=1
Test 9: A: 0000000000000001 B: 0000000000000001 Opcode: 0010 Result: 0000001000000000 Flags: C=0, D=0, Z=0
Test 10: A: 0000000000000001 B: 0000000000000000 Opcode: 0010 Result: 0000000000000001 Flags: C=0, D=0, Z=0
Test 11: A: 0023185d0fb1810 B: fff2308475f6309 Opcode: 0010 Result: 0023185d0fb1810 Flags: C=1, D=0, Z=0
Test 12: A: 0000000000000000 B: ffffffff00000000 Opcode: 0010 Result: 0000000000000000 Flags: C=1, D=0, Z=1
Test 13: A: 0000000000000000 B: 0000000000000000 Opcode: 0110 Result: 0000000000000000 Flags: C=0, D=1, Z=1
Test 14: A: 0000000000000001 B: 0000000000000001 Opcode: 0110 Result: 0000000000000000 Flags: C=0, D=1, Z=1
Test 15: A: 0000000000000001 B: 0000000000000001 Opcode: 0110 Result: ffffffff00000000 Flags: C=0, D=1, Z=0
Test 16: A: 0000000000000000 B: 0000000000000001 Opcode: 0110 Result: ffffffff00000000 Flags: C=0, D=1, Z=0
Test 17: A: 7fffffff00000000 B: ffffffff00000000 Opcode: 0110 Result: 0000000000000000 Flags: C=1, D=1, Z=0
Test 18: A: 0000000000000000 B: 0000000000000000 Opcode: 0110 Result: 0000000000000000 Flags: C=1, D=1, Z=0
Test 19: A: ffffffff00000000 B: ffffffff00000000 Opcode: 0110 Result: 0000000000000000 Flags: C=0, D=1, Z=1
Test 20: A: 0000000000000001 B: 0000000000000001 Opcode: 0110 Result: 0000000000000001 Flags: C=0, D=0, Z=0
Test 21: A: 0023185d0fb1810 B: fff2308475f6309 Opcode: 0110 Result: 0023185d0fb1810 Flags: C=0, D=0, Z=0
Test 22: A: ffffffff00000000 B: 0000000000000001 Opcode: 0110 Result: ffffffff00000000 Flags: C=0, D=0, Z=0
Test 23: A: 0000000000000001 B: ffffffff00000000 Opcode: 0110 Result: 0000000000000001 Flags: C=0, D=0, Z=0
Test 24: A: ffffffff00000000 B: 0000000000000000 Opcode: 0000 Result: 0000000000000000 Zero=1
Test 25: A: 000020c1454777 B: 0112305060c309 Opcode: 0000 Result: 000020c1454777 Zero=0
Test 26: A: 5555555555555555 B: aaaaaaaaaaaaaa Opcode: 0000 Result: 0000000000000000 Zero=1
Test 27: A: 123456789abcde0 B: ffffffff00000000 Opcode: 0000 Result: 123456789abcde0 Zero=0
Test 28: A: 0000000000000000 B: 0000000000000000 Opcode: 0001 Result: 0000000000000000 Zero=1
Test 29: A: ffffffff00000000 B: 0000000000000000 Opcode: 0001 Result: ffffffff00000000 Zero=0
Test 30: A: 5555555555555555 B: aaaaaaaaaaaaaa Opcode: 0001 Result: ffffffff00000000 Zero=0
Test 31: A: 000020c1454777 B: 0112305060c309 Opcode: 0001 Result: 000020c1454777 Zero=0
Test 32: A: 0000000000000000 B: 0000000000000000 Opcode: 0111 Result: 0000000000000000 Zero=1
Test 33: A: 5555555555555555 B: aaaaaaaaaaaaaa Opcode: 0111 Result: ffffffff00000000 Zero=0
Test 34: A: 5555555555555555 B: aaaaaaaaaaaaaa Opcode: 0111 Result: ffffffff00000000 Zero=0
Test 35: A: 123456789abcde0 B: 123456789abcde0 Opcode: 0111 Result: 0000000000000000 Zero=1
Test 36: A: 0000000000000001 B: 0000000000000000 Opcode: 0011 Result: 0000000000000001 Zero=0
Test 37: A: 0000000000000001 B: 0000000000000000 Opcode: 0011 Result: 0000000000000000 Zero=0
Test 38: A: aaaaaaaaaaaaaa B: 0000000000000001 Opcode: 0011 Result: 5555555555555555 Zero=0
Test 39: A: 0000000000000000 B: 0000000000000000 Opcode: 0011 Result: 0000000000000000 Zero=1
Test 40: A: 0000000000000000 B: 0000000000000000 Opcode: 1000 Result: 0000000000000000 Zero=0
Test 41: A: 7000000000000000 B: 0000000000000000 Opcode: 1000 Result: 0000000000000000 Zero=1
Test 42: A: 8000000000000000 B: 0000000000000000 Opcode: 1000 Result: 0000000000000000 Zero=0
Test 43: A: ffffffff00000000 B: 0000000000000000 Opcode: 1000 Result: 0000000000000000 Zero=0
Test 44: A: 8000000000000000 B: 0000000000000000 Opcode: 1010 Result: c000000000000000 Zero=0
Test 45: A: 4000000000000000 B: 0112305060c309 Opcode: 1010 Result: 2000000000000000 Zero=0
Test 46: A: 0000000000000001 B: 0000000000000001 Opcode: 1010 Result: 0000000000000000 Zero=1
Test 47: A: 8000000000000000 B: 0000000000000000 Opcode: 1010 Result: 0000000000000000 Zero=0
Test 48: A: 0000000000000000 B: 0000000000000000 Opcode: 0100 Result: 0000000000000000 Zero=1
Test 49: A: ffffffff00000000 B: 0000000000000000 Opcode: 0100 Result: 0000000000000000 Zero=0
Test 50: A: 0000000000000000 B: ffffffff00000000 Opcode: 0100 Result: 0000000000000000 Zero=1
Test 51: A: 0000000000000000 B: ffffffff00000000 Opcode: 0100 Result: 0000000000000001 Zero=0
Test 52: A: 0000000000000000 B: 0000000000000000 Opcode: 0101 Result: 0000000000000000 Zero=1
Test 53: A: 0000000000000001 B: 0000000000000000 Opcode: 0101 Result: 0000000000000000 Zero=1
Test 54: A: 7fffffff00000000 B: 0000000000000000 Opcode: 0101 Result: 0000000000000001 Zero=0
Test 55: A: ffffffff00000000 B: ffffffff00000000 Opcode: 0101 Result: 0000000000000000 Zero=1

FINAL RESULT: Passed 55/55 tests

alu_tb.v:295: $finish called at 500000 (ps)

```

Figure 14: Sample Testbench output of ALU



Figure 15: GTKwave of ALU

Stage 4: Memory Access

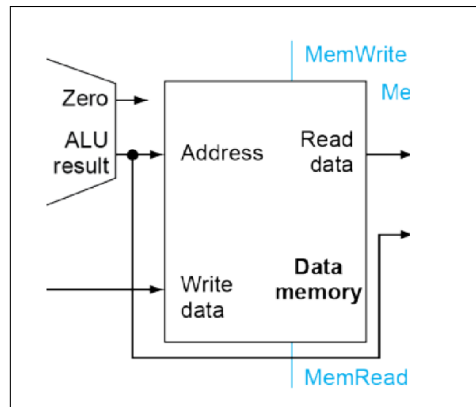


Figure 16: Memory access

Memory Access Behavior

Load Instructions (LD)

- The effective memory address is computed as the *ALU result*.
- The processor reads 64 bits from Data Memory at the computed address.
- The data becomes available at the beginning of the next clock cycle.
- This value is written back into the destination register during the Write-Back stage.

Store Instructions (SD)

- The effective memory address is computed as the *ALU result*.
- The processor writes 64 bits from a source register into Data Memory at this address.
- No value is written back into the register file.
- The next instruction proceeds without any additional write-back activity.

All Other Instructions

- The Memory Access stage performs no operation.
- The ALU output is forwarded directly to the Write-Back multiplexer.

Role of MemRead and MemWrite

These control signals originate from the Control Unit based on the instruction's opcode:

- **MemRead** = 1: The instruction requires reading from Data Memory (e.g., `ld`).
- **MemWrite** = 1: The instruction requires writing to Data Memory (e.g., `sd`).

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
<code>ld</code>	1	1	1	1	0	0	0	0
<code>sd</code>	1	X	0	0	1	0	0	0
<code>beq</code>	0	X	0	0	0	1	0	1

Figure 17: opcodes of the Instructions

Verification:-

The module has been implemented in verilog and has been verified with an appropriate test bench. The output verifying the test-cases is as follows:-

```

-----Data Memory Tests-----
PASS address=000000000 write_data=deadbeefcafeba MemRead=0 MemWrite=1 read_data=0000000000000000
PASS address=000000000 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=deadbeefcafeba
PASS address=000000100 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=0000000000000000
PASS address=000000100 write_data=1234567890abcdef MemRead=0 MemWrite=1 read_data=0000000000000000
PASS address=000000000 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=deadbeefcafeba
PASS address=000000100 write_data=0000000000000000 MemRead=1 MemWrite=0 read_data=1234567890abcdef
PASS address=000000000 write_data=ffffffffffffff MemRead=1 MemWrite=0 read_data=deadbeefcafeba
All tests passed!
data_mem_tb.v:88: $finish called at 72000 (1ps)

```

Figure 18: Sample Testbench output of Data Memory module



Figure 19: GTKwave of Data Memory module

Stage 5: Write Back

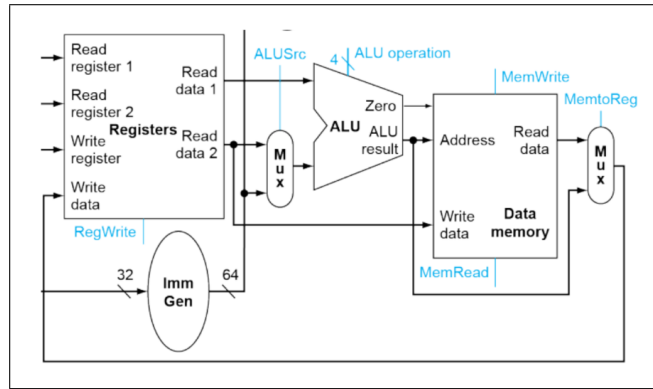


Figure 20: Write back into the register file

- The final output of an instruction is written to the Register File.
 - For ALU instructions, the value produced by the ALU is written to the destination register.
 - For load instructions, the value retrieved from Data Memory is written back instead.
- A multiplexer at the end of the datapath selects whether the Write-Back stage receives the ALU result or the memory output.
- The **RegWrite** signal generated by the Control Unit enables the register file to store the selected value.

Implementation of Fibonacci Sequence Code

We tested our code with a simple iterative version of the first ten elements of the Fibonacci series. The assembly code for implementing this is as shown below.

```

addi x1, x0, 10      /* Calculate 10 Fibonacci numbers*/
addi x2, x0, 0        /* Address for storing results*/
addi x3, x0, 0        /* Fib(0) = 0*/
addi x4, x0, 1        /* Fib(1) = 1*/
sd x3, 0(x2)         /* Store Fib(0)*/
addi x2, x2, 8        /* Increment address (changed from 8)*/
sd x4, 0(x2)         /* Store Fib(1)*/
addi x2, x2, 8        /* Increment address (changed from 8)*/
addi x5, x0, 2        /* i = 2*/
loop:
    beq x5, x1, done
  
```

```

        add x6, x3, x4
        sd x6, 0(x2)
        addi x2, x2, 8
        addi x5, x5, 1
        add x3, x0, x4
        add x4, x0, x6
        beq x0, x0, loop
done:
        add x2, x0, x0          /*iterating address for loading*/
        ld x10, 0(x2)          /*put the series onto the registers (x10–x19)*/
        ld x11, 8(x2)
        ld x12, 16(x2)
        ld x13, 24(x2)
        ld x14, 32(x2)
        ld x15, 40(x2)
        ld x16, 48(x2)
        ld x17, 56(x2)
        ld x18, 64(x2)
        ld x19, 72(x2)

```

At the end of the code we are expected to see registers 10-19 be filled with the first 10 values of the Fibonacci series, which is seen in the output file as follows:

```

0000000000000000
0000000000000000a
0000000000000000
000000000000000015
000000000000000022
0000000000000000a
000000000000000022
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
00000000000000001
00000000000000001
00000000000000002
00000000000000003
00000000000000005
00000000000000008
0000000000000000d
000000000000000015
000000000000000022
0000000000000000
0000000000000000
0000000000000000

```

```

0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000
0000000000000000

```

86

The waveform of different control signals, instruction and program counter are plotted to show an instant where the code takes a branch and executes the same set of instructions again is shown below:

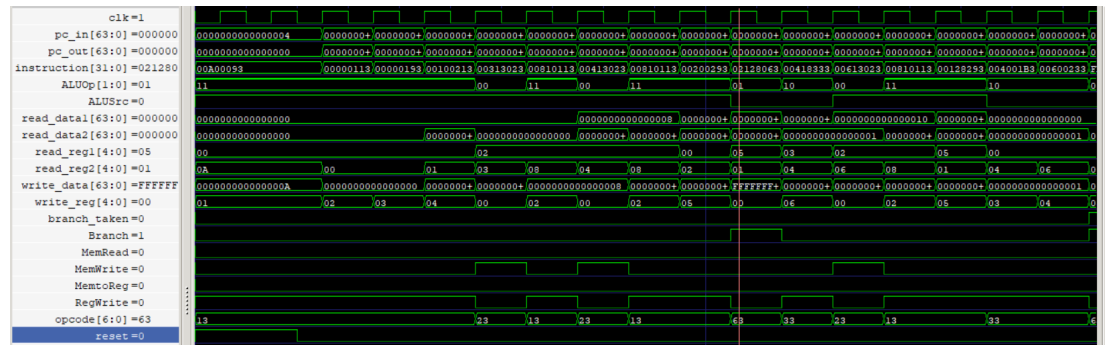


Figure 21: Waveform showing an instant of execution of the fibonacci program.

This concludes our processor's design and verification of the same.